

ノートブック実験環境 入門

Jupyter と marimo — 使い方・使い分け・道具を深く知ること

complex_systems プロジェクト 講義資料 #01

作成: 2026-06-21

0. この資料のねらい

複雑系の実験は「式を書く → 数値計算する → 図にする → 気づく」の反復だ。この反復を回す主戦場がノートブック環境である。本資料では二大ツール Jupyter と marimo の使い方と使い分けを扱う。

ただし真の目的はツールの操作習得そのものではない。「道具を1割しか使わない人」から「マニュアルを通読した上位数%」へ移行すること、そして自分の作業を観察して環境に落とし込む習慣を身につけることにある。プロの作業環境の差は単一の教科書からではなく、この習慣の堆積から生まれる。

1. なぜノートブックか

通常のスクリプト(.py を一括実行)と違い、ノートブックはセル単位で実行でき、コード・実行結果(数値や図)・文章(Markdown + 数式)を一枚の文書に混在させられる。探索的な実験 —

「このパラメータを変えると?」を即座に試して図で確かめる — に最適。

- ・ 対話性: 重い計算を一度だけ走らせ、その結果を使って何度も可視化を試せる。
- ・ 物語性: 理論メモ・コード・図・考察を順番に並べ、後から読める研究ノートになる。
- ・ 共有性: 出力込みで保存すれば、実行しなくても GitHub 上で図が見える。

本プロジェクトでは 1 実験 = 1 ディレクトリ、その中にノートブックを置く運用とする。

2. Jupyter — 定番・計算してプロットする実験向き

2.1 起動と基本操作

```
pip install jupyter
jupyter notebook      # 旧来のUI
jupyter lab           # 多機能IDE風UI (推奨)
# 出力込みでコマンドラインから再実行 (再現性チェック):
jupyter nbconvert --to notebook --execute --inplace 実験.ipynb
```

2.2 セルの種類と必須ショートカット

セルは Code(コード) と Markdown(文章・数式) の2種類。コマンドモード(Escで入る・セルが青枠)と編集モード(Enterで入る)を切り替えて操作する。

ショートカット	動作
Shift + Enter	セルを実行して次へ
Ctrl + Enter	セルを実行してとどまる
Esc → A / B	上 / 下にセルを追加
Esc → D D	セルを削除
Esc → M / Y	Markdown / Code に変換
Esc → Z	操作を元に戻す

2.3 数式は Markdown セルに LaTeX で書く

```
$$x_{n+1} = r x_n(1-x_n)$$      # ブロック数式
インライン:  $\lambda > 0$  ならカオス。
```

2.4 Jupyter 最大のクセ：隠れた実行状態

Jupyter はセルを実行した順に状態(変数)が決まる。上から下を書いてあっても、実行順が違えば結果も違う。セルを消しても、その変数はカーネルのメモリに残り続ける。これが「自分の手元では動くのに再現できない」問題の主因。

対策の作法：

- ・ こまめに Kernel → Restart & Run All(全消去して上から再実行)で再現性を確認する。
 - ・ コミット前に必ず `nbconvert --execute` で「まっさらな状態から通る」ことを確かめる。
 - ・ セルは上から下に読めば再現できる順序で並べる。
-

3. marimo — リアクティブ・対話的に挙動を探る実験向き

3.1 思想：隠れ状態をなくす

marimo は Jupyter の「隠れた実行状態」問題を設計から排除した新しいノートブック。セル間の依存関係を自動で解析し、あるセルの変数を変えるとそれに依存するセルだけが自動で再実行される(表計算ソフトのようなリアクティブ性)。実行順序による不整合が原理的に起きない。

3.2 起動と保存形式

```
pip install marimo
marimo edit 実験.py      # 編集モードで起動
marimo run 実験.py      # Webアプリとして起動(コードを隠して操作だけ)
# 保存形式は .ipynb ではなく純粋な .py
```

保存が通常の .py である点が効く。git の差分がコードとして綺麗に出る(.ipynb は出力が JSON に混ざって差分が読みにくい)。普通の Python として import / 実行もできる。

3.3 強力な UI 要素(スライダー等)

スライダーやドロップダウンを数行で置き、動かすと依存セルが即再描画される。「結合強度 K を上げると、ある点で突然同期する」といった相転移を体感的に探るのに最適。

```
import marimo as mo
K = mo.ui.slider(0.0, 4.0, value=1.0, step=0.05, label='結合強度 K')
K                                     # ← これを表示するとスライダーが出る

# 別のセル: K.value を使うと、スライダーを動かす度に自動再実行
result = simulate(coupling=K.value)
plot(result)
```

3.4 marimo のクセ・制約

- 変数名はノートブック内で一意でなければならない(依存解析の代償)。
- セルを跨いだ変数の再代入は不可。関数やスコープで囲って整理する作法が要る。
- エコシステムは Jupyter ほど成熟していない(資料・拡張は Jupyter が豊富)。

4. 使い分け早見表

観点	Jupyter	marimo
向く実験	計算して図を出す (分岐図・重い解析)	パラメータを動かして 挙動を探る(相転移)
対話性	セルを手動で再実行	依存セルが自動で再実行 (リアクティブ)
再現性	実行順に注意が必要 (隠れ状態)	原理的に保証 (隠れ状態なし)
保存形式	.ipynb (JSON) 出力込みで図が残る	.py (純Python) git差分が綺麗
UI部品	ipywidgets(やや手間)	mo.ui.* (数行・標準)
成熟度・資料	非常に豊富	発展途上
アプリ化	不得手	marimo run で即Webアプリ

本プロジェクトでの方針

- ・ 計算結果を一度出して図にする系(実験01 分岐図など) → Jupyter。出力込みで保存すれば GitHub でそのまま図が見える。
- ・ パラメータをいじって挙動を体感する系(実験02 Kuramoto 同期など) → marimo。スライダーで臨界点を探る。
- ・ 迷ったら: 「動かして遊びたい」なら marimo, 「結果を記録して見せたい」なら Jupyter。

5. 実践チートシート

環境構築(このリポジトリ)

```
pip install -r requirements.txt
```

Jupyter

```
jupyter lab                # 起動
jupyter nbconvert --to notebook --execute ¥
    --inplace 実験.ipynb    # 出力込みで再実行
jupyter nbconvert --to html 実験.ipynb    # HTML書き出し
```

marimo

```
marimo edit 実験.py        # 編集
marimo run 実験.py         # アプリとして実行
marimo convert 旧.ipynb > 新.py    # Jupyter から変換
```

6. 次の一步と「道具を深く知る」習慣

次の実験02(Kuramoto 結合振動子)はあえて marimoで作る。スライダーで結合強度を上げ、バラバラだった振動子がある点で突然同期する様子を対話的に観察し、実験01(Jupyter)との手触りの違いを体で覚えるのが狙いだ。

最後に、本資料冒頭のねらいに戻る。プロの環境の差は買ってこられない。自分が1日に何度も繰り返している操作に気づき、それを楽にする手段を notes/ に書き留める — この堆積こそが、君だけの差別化された研究環境になる。今日その第一層として、ノートブックという道具の「未読マニュアル」を一冊読み終えた。